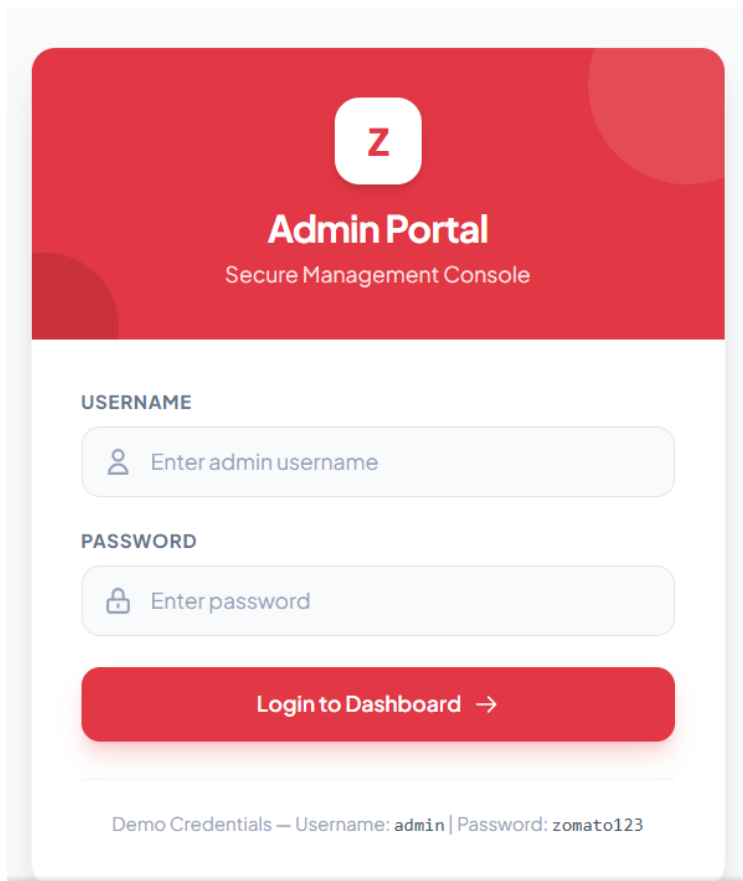


Day – 3 Authentication and Access Control.

Task – 1 Create login and password.



The image shows a login form for an 'Admin Portal'. At the top, there is a red header with a white rounded square containing a red 'Z' logo. Below the logo, the text 'Admin Portal' is written in white, followed by 'Secure Management Console' in a smaller white font. The main form area is white and contains two input fields. The first is labeled 'USERNAME' and has a placeholder 'Enter admin username' with a user icon. The second is labeled 'PASSWORD' and has a placeholder 'Enter password' with a lock icon. Below these fields is a red button with the text 'Login to Dashboard →'. At the bottom, there is a line of text: 'Demo Credentials — Username: admin | Password: zomato123'.

Task – 2 Python function.

-

```
def check_access
"""
Checks if a user role is authorized to access a specific resource.
"""
# Dictionary mapping roles to their allowed resources

"customer" "order_history"
"restaurant_owner" "menu_edit" "order_history"
```

```

    "admin" "menu_edit" "order_history" "admin_dashboard"

# Retrieve the allowed list for the role (defaults to empty list if role is unknown)

# Return True if the resource is in the allowed list, False otherwise
return        in

# --- Testing the function ---

"customer" "order_history"      # Expected: True
"customer" "menu_edit"          # Expected: False
"restaurant_owner" "menu_edit"  # Expected: True
"restaurant_owner" "admin_dashboard" # Expected: False
"admin" "admin_dashboard"       # Expected: True
"guest" "order_history"         # Expected: False (unknown role)

"Role Access Control Tests:"
"_" 50
for        in

    f"Role: {role:<18} | Resource: {resource:<18} | Access: {result}"

- permissions Dictionary: Maps each user role (key) to a list of strings representing the
  resources they are allowed to access (value).

- .get(user_role, []): Safely retrieves the allowed resources for the given role. If an
  unrecognized role is passed, it defaults to an empty list to prevent errors.

- in Operator: Checks if the requested resource exists within that role's allowed
  permissions list, returning a boolean value.

-

```

Task – 3 User to inshut OTP.

- import random
- def paytm_login_flow():
- # Hardcoded mock credentials
- correct_username = "user@paytm"
- correct_password = "securePassword123"
- print("--- Welcome to Paytm ---")
-

```

- # Step 1: Username and Password Verification
- entered_username = input("Enter your username/mobile: ")
- entered_password = input("Enter your password: ")
- if entered_username != correct_username or entered_password !=
correct_password:
-     print("\n[Error] Invalid username or password. Login Failed.")
-     return
- print("\nPassword verified successfully!")
-
- # Step 2: Generate a 6-digit OTP (simulating SMS dispatch)
- # Using random.randint to generate a number between 100000 and 999999
- generated_otp = str(random.randint(100000, 999999))
- print("\n--- SMS Notification ---")
- print(f"[SIMULATION] Your Paytm OTP is: {generated_otp}")
- print("-----")
-
- # Step 3: Prompt user for OTP input
- entered_otp = input("Enter the 6-digit OTP sent to your device: ").strip()
-
- # Step 4: Validate OTP
- if entered_otp == generated_otp:
-     print("\nLogin Successful! Welcome back to Paytm.")
- else:
-     print("\n[Error] Incorrect OTP. Access Denied.")
-
- # Run the simulation
- if __name__ == "__main__":
-     paytm_login_flow()

```

Task – 4 An array of user objects.

```

- from datetime import datetime
- def get_active_premium_users(users):
-     """
-     Returns users who are premium and have logged in within the last 7 days
-     relative to the fixed date '2024-06-10'.
-     """
-     today = datetime.strptime('2024-06-10', '%Y-%m-%d')
-     active_premium_users = []
-

```

```

-   for user in users:
-       # Check if the user is premium
-       if user.get("isPremium"):
-           last_login_date = datetime.strptime(user["lastLogin"], '%Y-%m-%d')
-           # Calculate the difference in days
-           days_difference = (today - last_login_date).days
-           # Check if login was within the last 7 days (0 to 7 days inclusive)
-           if 0 <= days_difference <= 7:
-               active_premium_users.append(user)
-   return active_premium_users
-
-   # --- Example Usage & Testing ---
-   sample_users = [
-       {"username": "alice", "isPremium": True, "lastLogin": "2024-06-08"}, # 2 days
-       # ago -> Included
-       {"username": "bob", "isPremium": True, "lastLogin": "2024-05-20"}, # Too old -
-       # > Excluded
-       {"username": "charlie", "isPremium": False, "lastLogin": "2024-06-09"}, # Not
-       # premium -> Excluded
-       {"username": "diana", "isPremium": True, "lastLogin": "2024-06-03"}, # Exactly
-       # 7 days ago -> Included
-       {"username": "eve", "isPremium": True, "lastLogin": "2024-06-11"} # Future
-       # date -> Excluded
-   ]
-   result = get_active_premium_users(sample_users)
-   print("Filtered Users:")
-   for u in result:
-       print(f"- {u['username']} (Last Login: {u['lastLogin']})")

```

Task – 5 Common mistake in application by developers.

- 1. Insecure Direct Object References (IDOR)
- The Mistake: Exposing direct references to database objects (such as changing an order ID or user profile ID in the URL parameters) allowing users to view data belonging to other accounts.
- Fix/Prevention Tip: Always validate on the server side whether the authenticated user has explicit ownership or permission to access the requested resource object.
-
- 2. Missing Function-Level Access Control

- The Mistake: Forgetting to apply authorization checks to backend administrative or merchant endpoints (such as /api/admin/delete-product), assuming that obfuscating the URL is enough.
- Fix/Prevention Tip: Enforce strict role-based access control (RBAC) middleware on every sensitive route to verify user privileges before executing any action.
-
- 3. Relying on Client-Side Access Control
- The Mistake: Hiding administrative or restricted UI buttons (like a delete icon or an edit menu button) on the frontend while leaving the underlying API endpoints completely unprotected.
- Fix/Prevention Tip: Treat the frontend as entirely untrusted and duplicate all security and authorization checks on the server side.
-
- 4. Horizontal Privilege Escalation via Parameter Tampering
- The Mistake: Trusting user-supplied identifiers inside request bodies or headers (like passing a different user handle in a POST request) to perform actions on behalf of another user at the same privilege level.
- Fix/Prevention Tip: Always extract and use the user identity directly from the secure session token or authentication context rather than accepting it from request parameters.
-
- 5. Overly Permissive Cross-Origin Resource Sharing (CORS)
- The Mistake: Configuring APIs with wildcard headers (Access-Control-Allow-Origin: *) alongside credential allowances, which lets malicious external websites read private user data via cross-site requests.
- Fix/Prevention Tip: Explicitly whitelist trusted domain names rather than using wildcards whenever an API endpoint handles sensitive user sessions or cookies.
-

